2-3 · MongoDB 구축 및 비정형 로그 적재 · 실습 1.5H

비정형 로그 데이터 적재와 집계 조회

1,000건을 넣고 집계하고 인덱스 전후를 재 본다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

실습 목표

1.5시간 뒤에는

① .venv 가상환경에 pymongo 를 설치하고 접속한다

② load_logs.py 로 접속 로그 1,000건을 insert_many 한다

③ $match·$group·$sort 로 레벨별·경로별·분 단위 집계를 만든다

④ 인덱스 생성 전후의 totalDocsExamined 를 직접 재서 비교한다

재 보지 않으면 효과를 알 수 없습니다 — 인덱스는 만들기 전에 먼저 재고, 만든 뒤 다시 잽니다

흐름

다섯 단계와 미니 과제

적재는 파이썬, 집계와 인덱스는 mongosh 에서 — 확인 포인트마다 숫자를 맞춰 봅니다

1. 환경	2. 적재	3. 집계	4. 시계열	5. 인덱스
.venv	load_logs.py	레벨별	$dateToString	explain
pymongo	1,000건	경로별	분 단위	전후 비교

1단계

파이썬 환경 — 가상환경부터

1 **mkdir -p ~/data-eng/week2 · cd ~/data-eng/week2**
1주차에 만든 폴더 구조를 이어 쓴다

2 **python3 -m venv .venv · source .venv/bin/activate**
프로젝트마다 라이브러리를 따로 두는 장치 — 프롬프트 앞에 (.venv) 가 붙는다

3 **pip install pymongo**
버전 4.18.2 · sys.prefix 가 .../week2/.venv 로 끝나면 준비 완료

나가기 · 다시 들어오기

deactivate

source ...activate

우분투·WSL2 는 python3-venv 먼저 · type python3 가 alias 면 unalias 뒤 hash -r — (.venv) 가 붙어도 sys.prefix 가 다르면 가상환경을 건너뛴 것(실측)

2단계

load_logs.py 가 하는 일

MongoClient	자기 문서 지우기	1,000건 생성	insert_many
localhost:27017 logdb · access_logs	source: load_logs delete_many	random.seed(42) ts 3초 간격	한 번에 적재 inserted_ids 개수 출력

시드 42 — 누가 돌려도 같은 1,000건 · ERROR 에만 error{code, message} 중첩 · INFO 가 자주 나오게 가중

deleted: 0 / inserted: 1000 · countDocuments() 1003 — 다시 들어갈 땐 mongosh logdb 로(그냥 mongosh 면 test DB 라 0)

일부러 겪어 보기

두 번 돌려도 1003 — 먼저 지우고 다시 넣는다

지우는 줄이 없으면

두 번째에 1,000건이 또 들어가 2003건(실측)

- 자동 _id 는 매번 새 값 — 중복 검사가 없다
- 오류 없이 집계 숫자만 조용히 두 배

VS

이 스크립트는

source 표시로 자기 문서만 지우고 다시 넣는다

- 두 번째: deleted: 1000 / inserted: 1000 → 1003
- 몇 번 돌려도 같은 결과 — 멱등(idempotent)

3·4단계

집계 다섯 가지 — 파이프라인과 확인 포인트

하려는 일	파이프라인	확인
레벨별 건수	$group _id: '$level' → $sort	INFO 587 · WARN 214 · ERROR 202
ERROR 경로 순위	$match ERROR → $group '$path' → $sort	/api/pay 47 이 1위
두 값으로 묶기	_id: { level: '$level', path: '$path' } → $limit 5	(INFO, /login) 125가 1위
경로별 평균	$avg: '$status' · $sum: 1	/api/pay 267.1 이 최고
분 단위 시계열	$dateToString '%Y-%m-%d %H:%M' · $cond	09:00 은 cnt 20 · errors 5

건수가 같으면 순서가 바뀔 수 있어 $sort 에 _id: 1 을 보조 기준으로(2-2 동점과 같은 이유) · $cond 는 CASE WHEN

5단계

인덱스 — 재 보고 판단한다

1. **만들기 전 — explain('executionStats')**
   totalDocsExamined 1003 — 전체 문서를 훑는다

2. **createIndex({ level: 1, ts: -1 })**
   getIndexes() 로 생겼는지 확인 · 1=오름차순, -1=내림차순

3. **만든 뒤 — 다시 측정**
   ERROR 건수 202만 훑으면 성공 — 줄지 않았다면 인덱스를 안 탄 것, 필드 이름을 다시

복합 인덱스 규칙

왼쪽부터

이어 쓴 만큼만

> **{ level: 1, ts: -1 } 은 level 로 거르고 ts 로 정렬하는 쿼리엔 맞지만 ts 만으로 거르는 쿼리엔 쓰이지 않는다 — MySQL 인덱스도 같은 규칙**

6단계

미니 과제 — 세 가지

01

사용자별 상위 5명

$group _id: '$user'

$sort → $limit 5

02

에러 코드별 건수

_id: '$error.code'

중첩 필드는 문자열로

03

$match 를 뒤로

$group 뒤로 옮기면

결과가 빈다 — 왜?

상위 5명 29·29·29·29·28 · 에러 코드 E80 8건이 1위 · 셋째는 빈 결과 — $group 뒤엔 level 이 없다

확인 포인트

여기까지 됐으면 성공

1. (.venv) · pymongo 4.18.2 · sys.prefix 가 .../.venv
2. deleted: 0 / inserted: 1000 · 두 번 돌려도 1003
3. INFO 587 · WARN 214 · ERROR 202, ERROR 경로 1위 /api/pay 47
4. 분 단위 10행 — 09:00 은 cnt 20 · errors 5
5. totalDocsExamined 1003 → 202

숫자가 안 맞으면 필드 이름부터 — 스키마가 없어 오타를 DB 가 잡아 주지 않습니다

2-3 · 실습 1.5H 마무리

다음은 정리·회고 (0.5H)

mongosh 명령을 SQL 과 나란히 정리하고, 2-4 에 쓸 컨테이너를 확인합니다

- ✓ **mongosh ↔ SQL 대응표**
- ✓ **오늘의 체크리스트**
- ✓ **자주 겪는 오류 네 가지**
- ✓ **2-4 ERD 설계 예고**

가져올 것

실습 중 막혔던 지점 메모